

Exp 6:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
wine = load_wine()
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("KNN Performance on Wine Dataset")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
KNN Performance on Wine Dataset
-----
Training samples: 142
Testing samples : 36
Accuracy: 0.7222222222222222
```

Classification Report:

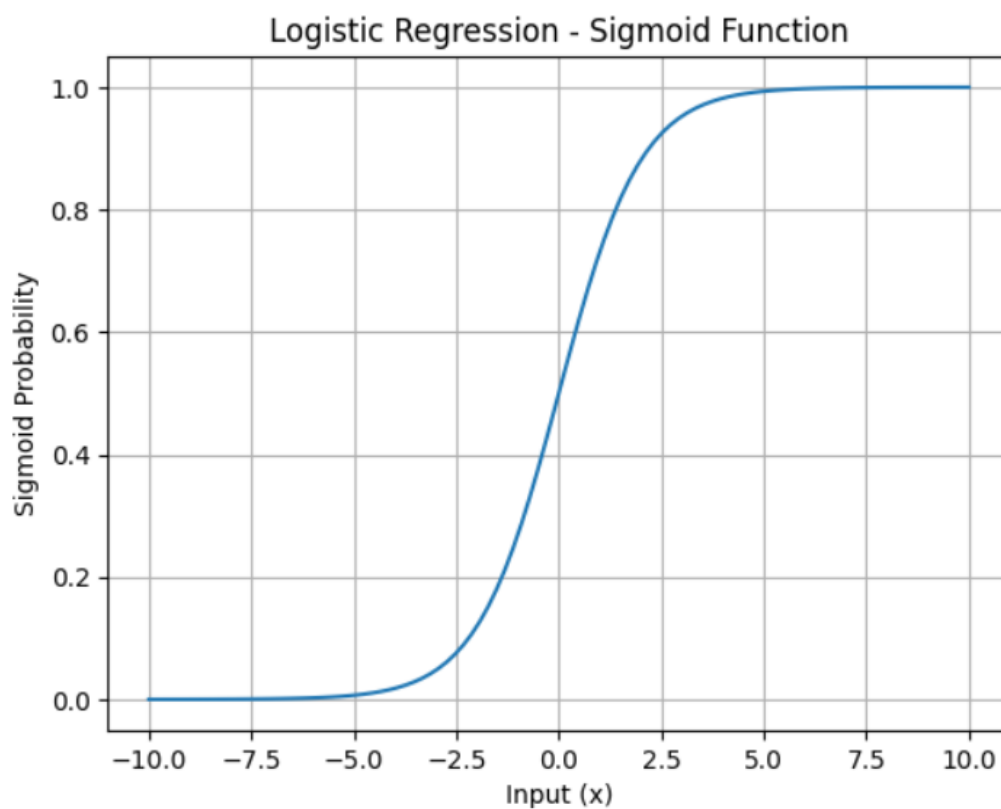
	precision	recall	f1-score	support
0	0.86	0.86	0.86	14
1	0.79	0.79	0.79	14
2	0.38	0.38	0.38	8
accuracy			0.72	36
macro avg	0.67	0.67	0.67	36
weighted avg	0.72	0.72	0.72	36

EXP 7

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-10, 10, 100)
y = 1 / (1 + np.exp(-x))
plt.plot(x, y)
plt.title("Logistic Regression - Sigmoid Function")
plt.xlabel("Input (x)")
plt.ylabel("Sigmoid Probability")
plt.grid(True)
plt.show()
```

OUTPUT:

...



Exp 8:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("KNN Performance on Iris Dataset")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)
```

OUTPUT

```
KNN Performance on Iris Dataset
```

```
-----
```

```
Training samples: 120
```

```
Testing samples : 30
```

```
Accuracy: 1.0
```

```
Confusion Matrix:
```

```
[[10  0  0]
```

```
 [ 0  9  0]
```

```
 [ 0  0 11]]
```

EXP 9

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Naive Bayes Performance on Iris Dataset")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT

```
Naive Bayes Performance on Iris Dataset
-----
Training samples: 120
Testing samples : 30
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

EXP 10

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Logistic Regression Performance on Iris Dataset")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT

```
Logistic Regression Performance on Iris Dataset
-----
Training samples: 120
Testing samples : 30
Accuracy: 1.0

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00         10
     1           1.00        1.00        1.00          9
     2           1.00        1.00        1.00         11

   accuracy                   1.00          30
  macro avg           1.00        1.00        1.00          30
 weighted avg           1.00        1.00        1.00          30
```